

Práctica 1: Tu Primer Asistente con Herramientas (Function Calling)

Introducción

Los Grandes Modelos de Lenguaje (LLMs) como GPT-4 son excelentes conversando, pero no pueden acceder a datos privados o en tiempo real por sí mismos.

En esta práctica, convertiréis un chat normal en un **Asistente de Aerolínea (FlightAI)** capaz de consultar una "base de datos" de precios interna usando una técnica llamada **Function Calling**.

Objetivos

1. Crear una función en Python que simule una consulta de base de datos.
2. Describir esa función en formato JSON para que la IA la entienda.
3. Programar el "manejador" que ejecuta el código cuando la IA lo solicita.

Pasos de la Práctica

Paso 1: La Base de Datos y la Función

Lo primero es tener datos. Copiad este diccionario que actúa como nuestra base de datos de precios y la función para leerla.

Añadid esto a vuestro código (sección de herramientas):

Python

1. Base de datos ficticia de precios

```
ticket_prices = {  
    "london": "$799",  
    "paris": "$899",  
    "tokyo": "$1400",  
    "berlin": "$499"  
}
```

2. Función Python real para obtener el precio

```
def get_ticket_price(destination_city):  
    print(f"🔧 Tool get_ticket_price called for {destination_city}") # Para depuración  
    city = destination_city.lower()  
    return ticket_prices.get(city, "Unknown")
```

Paso 2: Enseñar la Herramienta al LLM (Schema)

El LLM no puede "ver" vuestro código Python. Necesitamos describirle la función usando un formato específico (JSON).

Añadid esta definición a vuestro código:

Python

3. Definición de la herramienta para OpenAI

```
price_function = {  
    "name": "get_ticket_price",  
    "description": "Get the price of a return ticket to the destination city. Call this whenever  
you need to know the ticket price.",  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "destination_city": {  
                "type": "string",  
                "description": "The city that the customer wants to travel to",  
            },  
        },  
        "required": ["destination_city"],  
        "additionalProperties": False  
    }  
}
```

Lista de herramientas habilitadas

```
tools = [{"type": "function", "function": price_function}]
```

Paso 3: El Manejador (Handle Tool Call)

Cuando la IA decida usar la herramienta, pausará la conversación y nos pedirá que ejecutemos la función. Necesitamos una función intermedia (handle_tool_call) que haga el trabajo sucio.

Implementad esta función:

Python

```
def handle_tool_call(message):
```

```
    # 1. Obtener qué pide la IA
```

```

tool_call = message.tool_calls[0]

# 2. Decodificar los argumentos (ej: {"destination_city": "Berlin"})
arguments = json.loads(tool_call.function.arguments)
city = arguments.get('destination_city')

# 3. Ejecutar nuestra función Python (Paso 1)
price = get_ticket_price(city)

# 4. Preparar la respuesta para la IA
response = {
    "role": "tool",
    "content": json.dumps({"destination_city": city, "price": price}),
    "tool_call_id": tool_call.id
}

return response, city

```

Paso 4: Integración en el Chat

Aseguraos de que vuestra función chat incluye el parámetro tools en la llamada a OpenAI y maneja la respuesta.

Nota: Si estáis usando la plantilla base proporcionada por el profesor, esta parte ya debería estar estructurada, solo tenéis que aseguráros de que las variables coincidan.

Comprobación y Pruebas

Una vez tengáis el código funcionando, debéis realizar estas **3 pruebas obligatorias** para dar la práctica por válida:

1. Prueba de Éxito (Dato existente)

Preguntad por una ciudad que **sí** esté en vuestro diccionario (London, Paris, Tokyo, Berlin).

Usuario: "How much is a ticket to Berlin?"

Comportamiento esperado:

- La terminal muestra: Tool get_ticket_price called for Berlin
- **El Chatbot responde:** "A return ticket to Berlin costs \$499."

2. Prueba de "Dato No Disponible" (Manejo de errores)

Preguntad por una ciudad que **no** esté en la lista (por ejemplo, Madrid o New York).

Usuario: "How much is a ticket to Madrid?"

Comportamiento esperado:

- La terminal muestra:  Tool get_ticket_price called for Madrid
- La función interna devuelve "Unknown".
- **El Chatbot responde:** Algo como "I'm sorry, I don't have price information for Madrid" o "Prices for Madrid are currently unavailable."
- *Nota: Es importante que la IA no se invente un precio.*

3. Prueba de Personalidad (System Prompt)

Preguntad al bot quién es para verificar que las instrucciones iniciales funcionan.

Usuario: "Who are you?"

Comportamiento esperado:

- No se llama a ninguna herramienta (no aparece nada nuevo en la terminal).
- **El Chatbot responde:** "I am a helpful assistant for an Airline called FlightAI."
- *Nota: Debe mencionar explícitamente "FlightAI" tal como definimos en la variable system_message.*